

CS T51 – OPERATING SYSTEMS**UNIT – II**

Threads: Overview – Threading issues - CPU Scheduling – Basic Concepts – Scheduling Criteria – Scheduling Algorithms – Multiple-Processor Scheduling – Real Time Scheduling - The Critical- Section Problem – Synchronization Hardware – Semaphores – Classic problems of Synchronization – Critical regions – Monitors.

2 Marks**1. Define throughput? (APR'14)**

- Throughput refers to the performance of tasks by a computing service or device over a specific period.
- It measures the amount of completed work against time consumed and may be used to measure the performance of a processor, memory and/or network communications.
- Number of processes completed per unit time.
- May range from 10 / second to 1 / hour depending on the specific processes.

2. Define race condition? (APR'14)

- A race condition is a condition of a program where its behavior depends on relative timing or interleaving of multiple threads or processes.
- Race condition is a situation where-
 - The final output produced depends on the execution order of instructions of different processes.
 - Several processes compete with each other.

3. Define Aging and starvation? (APR' 14)

- Starvation is a phenomenon in which a process that is present in the ready state and has low priority, keeps on waiting for the CPU allocation because some other process with higher priority comes with due respect to time.
- Aging is a technique of gradually increasing the priority of processes that wait in the system for a long time.

4. What are the benefits of multithreaded programming? (NOV'14))

- **Resource sharing:** As the threads can share the memory and resources of any process it allows any application to perform multiple activities inside the same address space.
- **Utilization of MultipleProcessor Architecture:** The different threads can run parallel on the multiple processors hence, this enables the utilization of the processor to a large extent and efficiency.
- **Reduced Context Switching Time:** The threads minimize the context switching time as in Thread Context Switching, the virtual memory space remains the same.

- **Economical:** The allocation of memory and resources during process creation comes with a cost. As the threads can distribute resources of the process it is more economical to create context-switch threads.

5. **What are the requirements that a solution to the critical section problem must satisfy? (May'16) (NOV'14)**

What are Critical Regions? List down various mechanisms used to deal with critical region problem. (NOV'16)

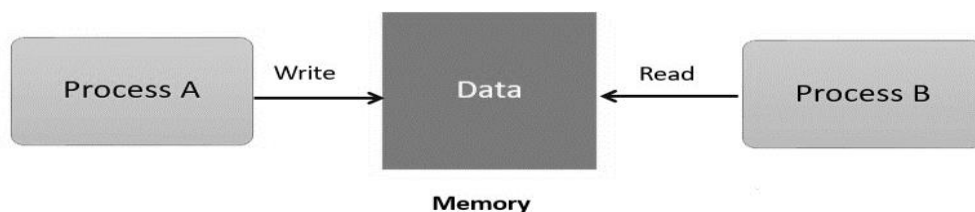
Define Critical Section [Nov 2018]. Explain with suitable example [May 2019]

- A critical region is an area of code where the code expects shared resources not to be changed or viewed by other threads while executing inside the critical region.
- Critical Section is the part of a program which tries to access shared resources. That resource may be any resource in a computer like a memory location, Data structure, CPU or any IO device.
- The critical section cannot be executed by more than one process at the same time;

Requirements / Solutions

- **Mutual Exclusion** - Mutual exclusion implies that only one process can be inside the critical section at any time. If any other processes require the critical section, they must wait until it is free.
- **Progress** -Progress means that if a process is not using the critical section, then it should not stop any other process from accessing it. In other words, any process can enter a critical section if it is free.
- **Bounded Waiting** -Bounded waiting means that each process must have a limited waiting time. It should not wait endlessly to access the critical section.

Example - Process A changing the data in a memory location while another process B is trying to read the data from the same memory location.



6. **What is a thread? (APR'15, NOV '15, May 18)**

Define the term thread [Nov 2019]

- A thread is a basic unit of CPU utilization, consisting of a program counter, a stack, and a set of registers, (and a thread ID.)
- Thread is a single sequence stream within a process.
- Threads are very useful in modern programming whenever a process has multiple tasks to perform independently of the others.
- They are called as light weight processes
- Types: Kernel Level Thread (KLT) ,User Level thread (ULT)

7. Define CPU scheduling (APR'15)

- In Multiprogramming systems, the Operating system schedules the processes on the CPU to have the maximum utilization of it and this procedure is called CPU scheduling.
- CPU scheduling is a process which allows one process to use the CPU while the execution of another process is on hold (in waiting state) due to unavailability of any resource like I/O etc, thereby making full use of CPU. The aim of CPU scheduling is to make the system efficient, fast and fair.
- Type: non-preemptive and preemptive Scheduling

8. What is a monitor? (NOV '15) [May 2019]

- Monitors are used for process synchronization.
- Monitors are defined as the construct of programming language, which helps in controlling shared data access.
- The Monitor is a module or package which encapsulates shared data structure, procedures, and the synchronization between the concurrent procedure invocations.
- Monitors are also often called conditional waits or conditional monitors.

9. Compare user threads and kernel threads? (May'16)

User Level Thread	Kernel Level Thread
User-level threads are faster to create and manage.	Kernel level threads are slower to create and manage.
Implemented by a thread_library user level.	Operating system support directly to Kernel threads.
User level thread can run on any operating system.	Kernel level threads are specific to the operating system.
Implementation of User threads is easy.	Implementation of Kernel thread is complicated.
Context switch time is less.	Context switch time is more.

10. Define Medium Term Scheduler (NOV'16)

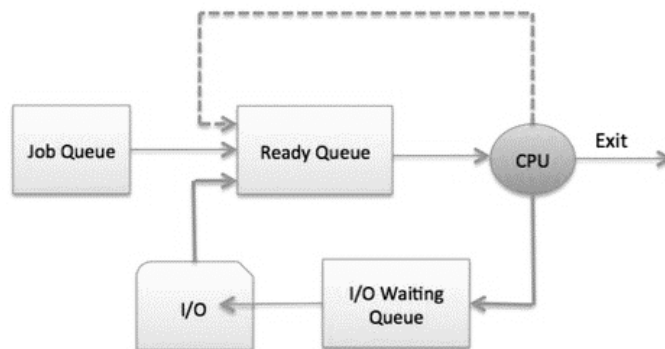
- Medium-term schedulers are those schedulers whose decision will have a mid-term effect on the performance of the system.
- It is responsible for swapping of a process from the Main Memory to Secondary Memory and vice-versa.
- It can re-introduce the process into memory and execution can be continued.
- Speed is in between both short and long term scheduler.
- A running process may become suspended if it takes an I/O request.
- A suspended process cannot make any progress towards completion.
- The main objective of medium term scheduler is to remove the process from the memory and create space for other processes; the suspended process is then moved to secondary storage.

11. What is the difference between process and thread? (May 2017)

Process	Thread
A process is a program under execution i.e an active program.	A thread is a lightweight process that can be managed independently by a scheduler.
Processes require more time for context switching as they are heavier.	Threads require less time for context switching as they are lighter than processes.
Processes are totally independent and don't share memory.	A thread may share some memory with its peer threads.
Processes require more resources than threads.	Threads generally need less resources than processes.
Processes have independent data and code segments.	A thread shares the data segment, code segment, files etc. with its peer threads.
All the different processes are treated separately by the operating system.	All user level peer threads are treated as a single task by the operating system.

12. What are the uses of job queues, ready queue and device queue? (MAY'17)

- **Job queue** – This queue keeps all the processes in the system.
- **Ready queue** – This queue keeps a set of all processes residing in main memory, ready and waiting to execute. A new process is always put in this queue.
- **Device queues** – The processes which are blocked due to unavailability of an I/O device constitute this queue.



13. What is Semaphore? [Nov 2017] [May 2018]

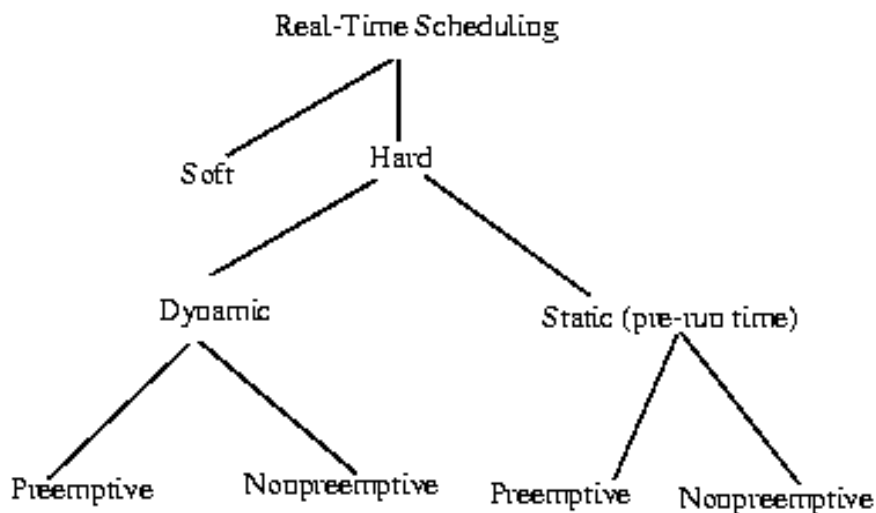
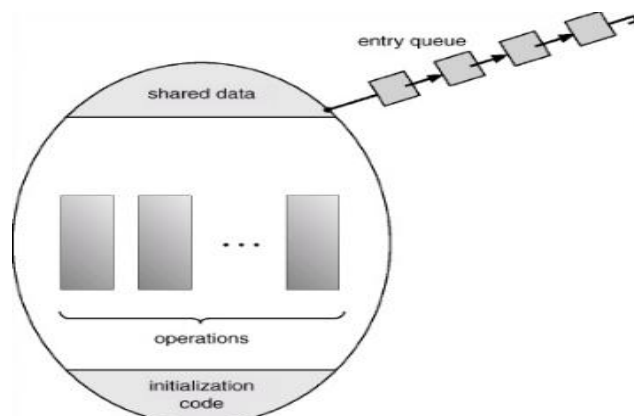
- A semaphore is a variable that is used to allow process synchronization.
- Semaphores work on a specific condition and allow the processor to communicate between multiple processes. A semaphore can have a value of 1 or 0.
- A semaphore has two operations:
- wait()- This operation decrements the value of the semaphore by 1, i.e., $s = s - 1$;
- signal()-This operation increments the value of the semaphore by 1, i.e., $s = s + 1$;

14. What are the different CPU Scheduling algorithms [Nov 2017]

- First Come First Serve (FCFS)
- Shortest-Job-First (SJF) Scheduling
- Shortest Remaining Time
- Priority Scheduling
- Round Robin Scheduling
- Multilevel Queue Scheduling

15. Write about Real time scheduling [Nov 2018]

- Real-time scheduling is more critical and difficult than traditional time-sharing.
- The scheduling problem is concerned with the allocation of the resources to satisfy the timing constraint
- Real-Time scheduling can be categorized into hard vs soft.
- A hard real-time system must execute a set of concurrent real-time tasks in a such a way that all time-critical tasks meet their specified deadlines. Every task needs computational and data resources to complete the job.
- Hard real-time scheduling can be used for soft real-time scheduling.

**16. Draw the Diagrammatic view of monitor [Nov 2019]**

17. What is fragmentation? [Sep 2020]

- Fragmentation is an unwanted problem where the memory blocks cannot be allocated to the processes due to their small size and the blocks remain unused.
- It can also be understood as when the processes are loaded and removed from the memory, they create free space or hole in the memory and these small blocks cannot be allocated to new upcoming processes and results in inefficient use of memory.
- Basically, there are two types of fragmentation:
 - Internal Fragmentation
 - External Fragmentation

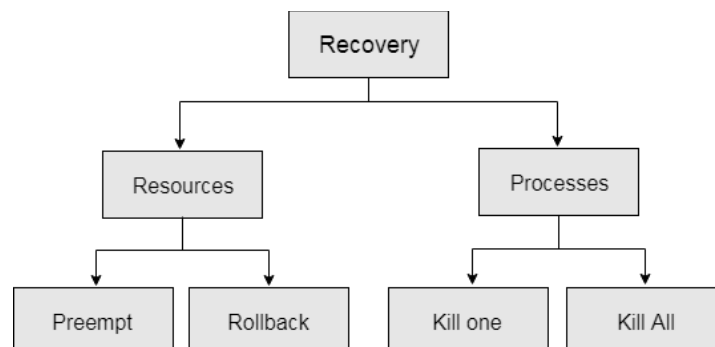
18. How does the system detect Thrashing [Sep 2020]

- Thrashing is caused by under allocation of the minimum number of pages required by a process, forcing it to continuously page fault.
- The system can detect thrashing by evaluating the level of CPU utilization as compared to the level of multiprogramming.
- It can be eliminated by reducing the level of multiprogramming.

PART 1 [11 Marks]**1.1 Discuss how to do recovery from deadlock [5] [Apr 2014]**

Deadlock recovery performs when a deadlock is detected. When deadlock detected, then our system stops working, and after the recovery of the deadlock, the system starts working again.

Therefore, after the detection of deadlock, a method/way must require to recover that deadlock to run the system again. The method/way is called as deadlock recovery. In order to recover the system from deadlocks, either OS considers resources or processes.

Various ways of deadlock recovery**1. Process Termination:**

This method of deadlock recovery through killing processes is the simplest way of deadlock recovery. Sometime it is best to kill a process that can be return from the beginning with no ill effects.

(a). Abort all the Deadlocked Processes:

- Aborting all the processes will certainly break the deadlock, but with a great expense.
- The deadlocked processes may have computed for a long time and the result of those partial computations must be discarded and there is a probability to recalculate them later.

(b). Abort one process at a time until deadlock is eliminated:

- Abort one deadlocked process at a time, until deadlock cycle is eliminated from the system.
- Due to this method, there may be considerable overhead, because after aborting each process, it is important to run a deadlock detection algorithm to check whether any processes are still deadlocked.

2. Resource Preemption:

- The ability to take a resource away from a process, have another process use it, and then give it back without the process noticing. It is highly dependent on the nature of the resource.
- Deadlock recovery through preemption is too difficult or sometime impossible. This method will raise three issues –

(a). Selecting a victim:

- In this method, it is first determined which resources and which processes are to be preempted and also the order to minimize the cost.

(b). Rollback:

- In this case of deadlock recovery through rollback, whenever a deadlock is detected, it is easy to see which resources are needed.
- To do the recovery of deadlock, a process that owns a needed resource is rolled back to a point in time before it acquired some other resource just by starting one of its earlier checkpoints.

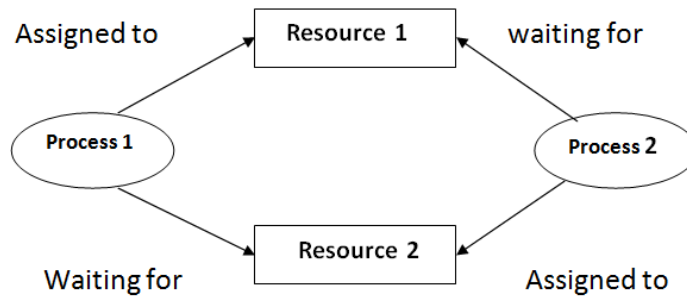
(c). Starvation:

- In a system, it may happen that same process is always picked as a victim. As a result, that process will never complete its designated task.
- This situation is called Starvation and must be avoided. One solution is that a process must be picked as a victim only a finite number of times.

1.2 Explain how deadlocks are handled in OS [Nov 2017]

Deadlock: In the multiprogramming operating system, there are a number of processes which fight for a finite number of resources and sometimes a waiting process never gets a chance to change its state because the resources for which it is waiting are held by another waiting process.

A set of processes is called a deadlock when they are waiting for the happening of an event which is called by some other event in the same set.



Here every process will follow the system model which means the process requests a resource if not allocated then wait otherwise it allocated will use the resources and release it after use.

Methods for handling deadlock - There are mainly four methods for handling deadlock.

1. Deadlock ignorance

- It is the most popular method and it acts as if no deadlock and the user will restart.
- As handling deadlock is expensive to be called of a lot of codes need to be altered which will decrease the performance so for less critical jobs deadlock are ignored. Ostrich algorithm is used in deadlock Ignorance.
- Used in windows, Linux etc.

2. Deadlock prevention

It means that we design such a system where there is no chance of having a deadlock.

- **Mutual exclusion:**
 - It can't be resolved as it is the hardware property.
 - For example, the printer cannot be simultaneously shared by several processes.
 - This is very difficult because some resources are not sharable.
- **Hold and wait:**
 - Hold and wait can be resolved using the conservative approach where a process can start it and only if it has acquired all the resources.
- **Active approach:**
 - Here the process acquires only requires resources but whenever a new resource requires it must first release all the resources.
- **Wait time out:**
 - Here there is a maximum time bound until which a process can wait for other resources after which it must release the resources.
- **Circular wait:**
 - In order to remove circular wait, we assign a number to every resource and the process can request only in the increasing order otherwise the process

must release all the high number acquires resources and then make a fresh request.

- **No pre-emption:**

- In no pre-emption, we allow forceful pre-emption where a resource can be forcefully pre-empted.
- The pre-empted resource is added to the list of resources where the process is waiting.
- The new process can be restarted only when it regains its old resources. Priority must be given to a process which is in waiting for state.

3. Deadlock avoidance

- Here whenever a process enters into the system it must declare maximum demand. To the deadlock problem before the deadlock occurs.
- This approach employs an algorithm to access the possibility that deadlock would occur and not act accordingly.
- If the necessary condition of deadlock is in place it is still possible to avoid feedback by allocating resources carefully.

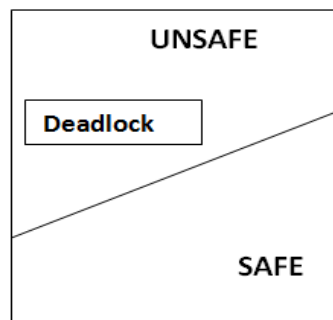
A **deadlock avoidance algorithm** dynamically examines the resources allocation state to ensure that a circular wait condition case never exists. Where the resources allocation state is defined by the of available and allocated resources and the maximum demand of the process.

There are 3 states of the system:

Safe state

When a system can allocate the resources to the process in such a way so that they still avoid deadlock then the state is called safe state. When there is a safe sequence exit then we can say that the system is in the safe state.

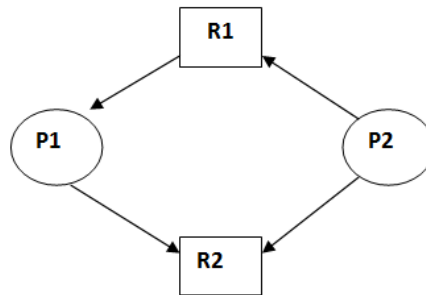
A sequence is in the safe state only if there exists a safe sequence. A sequence of process **P₁**, **P₂**, **P_n** is a safe sequence for the current allocation state if for each **P_i** the resources request that **P_i** can still make can be satisfied by currently available resources pulls the resources held by all **P_j** with **j < i**.



Methods for deadlock avoidance

1) Resource allocation graph

- This graph is also kind of graphical bankers' algorithm where a process is denoted by a circle P_i and resources is denoted by a rectangle R_j (.dots) inside the resources represents copies.
- Presence of a cycle in the resource's allocation graph is necessary but not sufficient condition for detection of deadlock. If the type of every resource has exactly one copy than the presence of cycle is necessary as well as sufficient condition for detection of deadlock.



This is in unsafe state (cycle exist) if P1 request P2 and P2 request R1 then deadlock will occur.

2) Bankers's algorithm

The resource allocation graph algorithms not applicable to the system with multiple instances of the type of each resource. So, for this system Banker's algorithm is used. Here whenever a process enters into the system it must declare maximum demand possible.

At runtime, we maintain some data structure like current allocation, current need, current available etc. Whenever a process requests some resources, we first check whether the system is in a safe state or not meaning if every process requires maximum resources then is there any sequence in which request can be entertaining if yes then request is allocated otherwise rejected.

Safety algorithm -This algorithm is used to find whether system is in safe state or not we can find:

Remaining Need = Max Need – Current allocation

Current available = Total available – Current allocation

Consider the following 3 process total resources are given for **A= 6, B= 5, C= 7, D = 6**

Process	maximum				Allocation				Need			
	A	B	C	D	A	B	C	D	A	B	C	D
P1	3	3	2	2	1	2	2	1	2	1	0	1
P2	1	2	3	4	1	0	3	3	0	2	0	1
P3	1	3	5	0	1	2	1	0	0	1	4	0

First we find the need matrix by **Need = maximum – allocation**. Then find available **resources = total – allocated**

A B C D (6 5 7 6) - A B C D (3 4 6 4), Available resources A B C D (3 1 1 2)

Then we check whether system is in deadlock or not & find safe sequence of process.

P1 can be satisfied: Available = P1 allocated + available (1,2,2,1) + (3,1,1,2) = (4,3,3,3)

P2 can be satisfied: Available = P2 allocated + available (1,0,3,3) + (4,3,3,3) = (5,3,6,6)

P3 can be satisfied: Available = P3 allocated + available (1,2,1,0) + (5,3,6,6) = (6,5,7,6)

So, the system is safe and the safe sequence is **P1 → P2 → P3**

4. Detection and recovery

When the system is in deadlock then one method is to inform the operator and then operator deal with deadlock manually and the second method is system will automatically recover from deadlock. There are two ways to recover from deadlock:

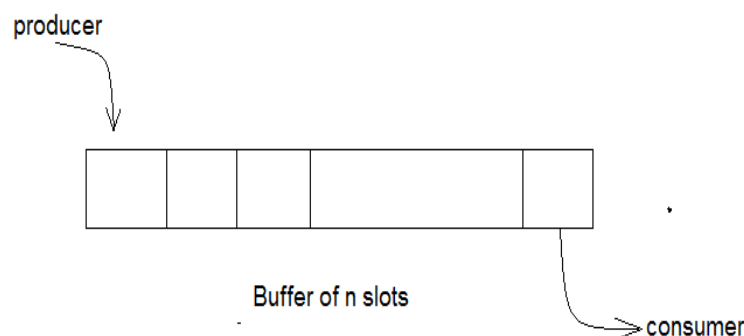
- **Process termination:** Deadlock can be eliminated by aborting a process. Abort all deadlock process. Abort is processed at a time until the deadlock cycle is eliminated. This can help to recover the system from file deadlock.
- **Resources preemption:** To eliminate deadlock using resources preemption, we prompt the same resources as processes and give these resources to another process until the deadlock cycle is broken. Here a process is partially rollback until the last checkpoint or when detection algorithm is executed.

1.3 Brief about Bounded-buffer problem [Apr 2014]

How Producer consumer problem can be solved using semaphore? Explain the solution with an example [Sep 2020]

Bounded buffer problem, which is also called **producer consumer problem**, is one of the classic problems of synchronization. Let's start by understanding the problem here, before moving on to the solution and program code.

Problem Statement: There is a buffer of n slots and each slot is capable of storing one unit of data. There are two processes running, namely, producer and consumer, which are operating on the buffer.



Bounded Buffer Problem

A producer tries to insert data into an empty slot of the buffer. A consumer tries to remove data from a filled slot in the buffer. Those two processes won't produce the expected output if they are being executed concurrently. There needs to be a way to make the producer and consumer work in an independent manner.

Solution: One solution of this problem is to use semaphores. The semaphores which will be used here are:

- **empty**, a **counting semaphore** whose initial value is the number of slots in the buffer, since, initially all slots are empty.
- **full**, a **counting semaphore** whose initial value is 0.

At any instant, the current value of **empty** represents the number of empty slots in the buffer and **full** represents the number of occupied slots in the buffer.

The Producer Operation: **The pseudocode of the producer function looks like this:**

```
do {  
    // wait until empty > 0 and then decrement 'empty'  
    wait(empty);  
    // acquire lock  
    wait(mutex);  
    /* perform the insert operation in a slot */  
    // release lock  
    signal(mutex);  
    // increment 'full'  
    signal(full);  
} while(TRUE)
```

- A producer first waits until there is atleast one empty slot.
- Then it decrements the **empty** semaphore because, there will now be one less empty slot, since the producer is going to insert data in one of those slots.
- Then, it acquires lock on the buffer, so that the consumer cannot access the buffer until producer completes its operation.
- After performing the insert operation, the lock is released and the value of **full** is incremented because the producer has just filled a slot in the buffer.

The Consumer Operation: **The pseudocode for the consumer function looks like this:**

```
do  
{  
    // wait until full > 0 and then decrement 'full'  
    wait(full);  
    // acquire the lock  
    wait(mutex);  
    /* perform the remove operation in a slot */  
    // release the lock  
    signal(mutex);  
    // increment 'empty'  
    signal(empty);  
} while(TRUE);
```

- The consumer waits until there is atleast one full slot in the buffer.

- Then it decrements the **full** semaphore because the number of occupied slots will be decreased by one, after the consumer completes its operation.
- After that, the consumer acquires lock on the buffer.
- Following that, the consumer completes the removal operation so that the data from one of the full slots is removed.
- Then, the consumer releases the lock.
- Finally, the **empty** semaphore is incremented by 1, because the consumer has just removed data from an occupied slot, thus making it empty.

1.4 Brief about reader-writer problem [6] [April 2014]

Write short notes on Readers-Writers Problem? (NOV'14)

The readers-writers problem is a classical problem of process synchronization, it relates to a data set such as a file that is shared between more than one process at a time. Among these various processes, some are Readers - which can only read the data set; they do not perform any updates, some are Writers - can both read and write in the data sets.

The readers-writers problem is used for managing synchronization among various reader and writer process so that there are no problems with the data sets, i.e. no inconsistency is generated.

The Problem Statement

- There is a shared resource which should be accessed by multiple processes.
- There are two types of processes in this context. They are reader and writer.
- Any number of readers can read from the shared resource simultaneously, but only one writer can write to the shared resource.
- When a writer is writing data to the resource, no other process can access the resource. A writer cannot write to the resource if there are non-zero number of readers accessing the resource at that time.

The Solution

- From the above problem statement, it is evident that readers have higher priority than writer. If a writer wants to write to the resource, it must wait until there are no readers currently accessing that resource.
- Here, we use one mutex *m* and a semaphore *w*. An integer variable *read_count* is used to maintain the number of readers currently accessing the resource. The variable *read_count* is initialized to 0. A value of 1 is given initially to *m* and *w*.
- Instead of having the process to acquire lock on the shared resource, we use the mutex *m* to make the process to acquire and release lock whenever it is updating the *read_count* variable.

The code for the writer process looks like this:

```
while(TRUE)
{
    wait(w);
    /* perform the write operation */
    signal(w);
}
```

And, the code for the reader process looks like this:

```
while(TRUE)
{
    //acquire lock
    wait(m);
    read_count++;
    if(read_count == 1)
        wait(w);
    //release lock
    signal(m);
    /* perform the reading operation */
    // acquire lock
    wait(m);
    read_count--;
    if(read_count == 0)
        signal(w);
    // release lock
    signal(m);
}
```

- As seen above in the code for the writer, the writer just waits on the w semaphore until it gets a chance to write to the resource.
- After performing the write operation, it increments w so that the next writer can access the resource.
- On the other hand, in the code for the reader, the lock is acquired whenever the read_count is updated by a process.
- When a reader wants to access the resource, first it increments the read_count value, then accesses the resource and then decrements the read_count value.
- The semaphore w is used by the first reader which enters the critical section and the last reader which exits the critical section.
- The reason for this is, when the first readers enter the critical section, the writer is blocked from the resource. Only new readers can access the resource now.
- Similarly, when the last reader exits the critical section, it signals the writer using the w semaphore because there are zero readers now and a writer can have the chance to access the resource.

1.5 Explain Synchronization and concurrency control [Nov 2014]

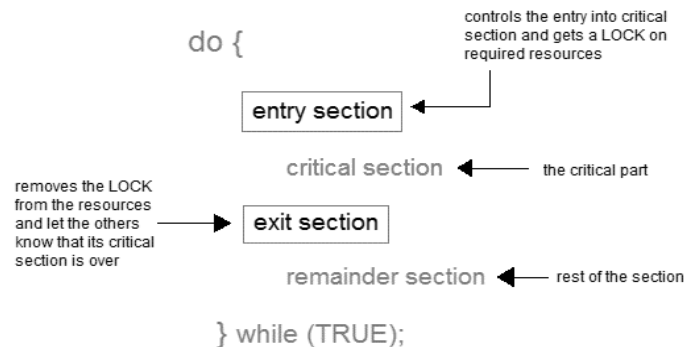
Synchronization

- Process Synchronization means sharing system resources by processes in a such a way that, Concurrent access to shared data is handled thereby minimizing the chance of inconsistent data. Maintaining data consistency demands mechanisms to ensure synchronized execution of cooperating processes.
- Process Synchronization was introduced to handle problems that arose while multiple process executions.

Critical Section Problem

A Critical Section is a code segment that accesses shared variables and has to be executed as an atomic action. It means that in a group of cooperating processes, at a given point of

time, only one process must be executing its critical section. If any other process also wants to execute its critical section, it must wait until the first one finishes.



Solution to Critical Section Problem -A solution to the critical section problem must satisfy the following three conditions:

- 1. Mutual Exclusion** - Out of a group of cooperating processes, only one process can be in its critical section at a given point of time.
- 2. Progress** -If no process is in its critical section, and if one or more threads want to execute their critical section then any one of these threads must be allowed to get into its critical section.
- 3. Bounded Waiting** - After a process makes a request for getting into its critical section, there is a limit for how many other processes can get into their critical section, before this process's request is granted. So, after the limit is reached, system must grant the process permission to get into its critical section.

Concurrency

- Concurrency is the execution of the multiple instruction sequences at the same time. It happens in the operating system when there are several process threads running in parallel.
- The running process threads always communicate with each other through shared memory or message passing. Concurrency results in sharing of resources result in problems like deadlocks and resources starvation.
- It helps in techniques like coordinating execution of processes, memory allocation and execution scheduling for maximizing throughput.
- Two processes are concurrent when they are executed in a way that their execution intervals overlap

Principles of Concurrency:Both interleaved and overlapped processes can be viewed as examples of concurrent processes, they both present the same problems. The relative speed of execution cannot be predicted. It depends on the following:

- The activities of other processes
- The way operating system handles interrupts
- The scheduling policies of the operating system

Problems in Concurrency:

- **Sharing global resources** –Sharing of global resources safely is difficult. If two processes both make use of a global variable and both perform read and write on that variable, then the order in which various read and write are executed is critical.
- **Optimal allocation of resources** –It is difficult for the operating system to manage the allocation of resources optimally.
- **Locating programming errors** –It is very difficult to locate a programming error because reports are usually not reproducible.
- **Locking the channel** –It may be inefficient for the operating system to simply lock the channel and prevents its use by other processes.

Advantages of Concurrency:

- **Running of multiple applications** –It enable to run multiple applications at the same time.
- **Better resource utilization** –It enables that the resources that are unused by one application can be used for other applications.
- **Better average response time** –Without concurrency, each application has to be run to completion before the next one can be run.
- **Better performance** –It enables the better performance by the operating system. When one application uses only the processor and another application uses only the disk drive then the time to run both applications concurrently to completion will be shorter than the time to run each application consecutively.

Drawbacks of Concurrency:

- It is required to protect multiple applications from one another.
- It is required to coordinate multiple applications through additional mechanisms.
- Additional performance overheads and complexities in operating systems are required for switching among applications.
- Sometimes running too many applications concurrently leads to severely degraded performance.

Issues of Concurrency:

- **Non-atomic** – Operations that are non-atomic but interruptible by multiple processes can cause problems.
- **Race conditions** – A race condition occurs if the outcome depends on which of several processes gets to a point first.
- **Blocking** – Processes can block waiting for resources. A process could be blocked for long period of time waiting for input from a terminal. If the process is required to periodically update some data, this would be very undesirable.
- **Starvation** – It occurs when a process does not obtain service to progress.
- **Deadlock** – It occurs when two processes are blocked and hence neither can proceed to execute.

1.5 Write short notes on Dining-Philosophers Problem (NOV'14)

Discuss related problems with reader-writers problem and the dining philosopher's problem [Nov 2014]

Give the solution for Dining-philosopher problem using monitors (NOV'16)

Outline the solution using semaphore to solve dining philosopher problem [Nov 2019]

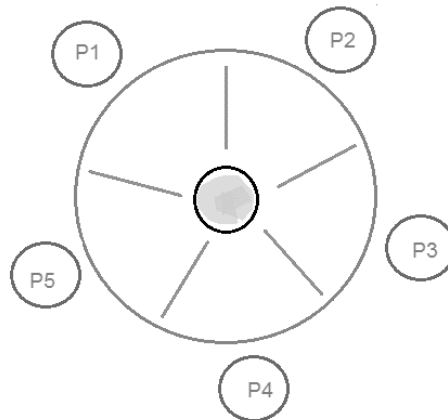
What is Semaphore? How do you solve dining philosopher problem using semaphore? [Nov 2018]

Dining Philosophers Problem

The dining philosophers problem is classic synchronization problem which is used to evaluate situations where there is a need of allocating multiple resources to multiple processes

Problem Statement

Consider there are five philosophers sitting around a circular dining table and their job is to think and eat alternatively. The dining table has five chopsticks and a bowl of rice in the middle as shown in the below figure.

**Dining Philosophers Problem**

At any instant, a philosopher is either eating or thinking. When a philosopher wants to eat, he uses two chopsticks - one from their left and one from their right. When a philosopher wants to think, he keeps down both chopsticks at their original place.

A bowl of noodles is placed at the centre of the table along with five chopsticks for each of the philosophers. To eat a philosopher needs both their right and a left chopstick. A philosopher can only eat if both immediate left and right chopsticks of the philosopher is available. In case if both immediate left and right chopsticks of the philosopher are not available then the philosopher puts down their (either left or right) chopstick and starts thinking again.

The dining philosopher demonstrates a large class of concurrency control problems hence it's a classic synchronization problem.

Solution

- From the problem statement, it is clear that a philosopher can think for an indefinite amount of time. But when a philosopher starts eating, he has to stop at some point of time. The philosopher is in an endless cycle of thinking and eating.
- An array of five semaphores, `stick [5]`, for each of the five chopsticks.

The code for each philosopher looks like:

```
while(TRUE)
{
    wait(stick[i]);
    /* mod is used because if i=5, next chopstick is 1 (dining table is circular) */
    wait(stick[(i+1) % 5]);
    /* eat */
    signal(stick[i]);
    signal(stick[(i+1) % 5]);
    /* think */
}
```

When a philosopher wants to eat the rice, he will wait for the chopstick at his left and picks up that chopstick. Then he waits for the right chopstick to be available, and then picks it too. After eating, he puts both the chopsticks down.

But if all five philosophers are hungry simultaneously, and each of them pickup one chopstick, then a deadlock situation occurs because they will be waiting for another chopstick forever. The possible solutions for this are:

- A philosopher must be allowed to pick up the chopsticks only if both the left and right chopsticks are available.
- Allow only four philosophers to sit at the table. That way, if all the four philosophers pick up four chopsticks, there will be one chopstick left on the table. So, one philosopher can start eating and eventually, two chopsticks will be available. In this way, deadlocks can be avoided.

Dining-Philosophers Solution Using Monitors

- A monitor is used to control access to state variables and condition variables.
- The fork and the spaghetti are not part of the monitor.
- Monitor procedures are defined for the actions of obtaining the forks and putting them down.
- These are used as entry and exit segments for program segments (here philosophers) which actually use the forks.
- A philosopher can be one of the three states THINKING, EATING, HUNGRY.
- For each philosopher, there is going to be condition variables where the philosopher will WAIT if he/she is hungry but one or both of the forks are unavailable.
- If a philosopher wants to eat, he will check the state of his neighbours and will eat if both his neighbours are not eating. Else he will wait.
- A philosopher who has finished eating will give his neighbour a chance to eat, if they are hungry and their other chopstick is free.

This solution imposes the restriction that a philosopher may pick up her chopsticks only if both of them are available.

- Consider following data structure:
 - `enum {THINKING, HUNGRY, EATING} state[5];`

- Philosopher i can set the variable $state[i] = \text{EATING}$ only if her two neighbors are not eating:
- $(state[(i+4) \% 5] \neq \text{EATING}) \text{ and } (state[(i+1) \% 5] \neq \text{EATING})$.
- And also declare: Condition $self[5]$;
- This allows philosopher i to delay herself when she is hungry but is unable to obtain the chopsticks she needs. A monitor solution to the dining-philosopher problem:

// Dining-Philosophers Solution Using Monitors

monitor DP

```
{
    status state[5];
    condition self[5];

    // Pickup chopsticks
    Pickup(int i)
    {
        // indicate that I'm hungry
        state[i] = hungry;

        // set state to eating in test() only if my left and right neighbors are not eating
        test(i);

        // if unable to eat, wait to be signaled
        if (state[i] != eating)
            self[i].wait;
    }

    // Put down chopsticks
    Putdown(int i)
    {
        // indicate that I'm thinking
        state[i] = thinking;

        // if right neighbor  $R=(i+1)\%5$  is hungry & both of R's neighbors are not eating,
        // set R's state to eating and wake it up by signaling R's CV
        test((i + 1) \% 5);
        test((i + 4) \% 5);
    }

    test(int i)
    {
        if (state[(i + 1) \% 5] != eating
            && state[(i + 4) \% 5] != eating
            && state[i] == hungry) {

            // indicate that I'm eating
            state[i] = eating;
        }
    }
}
```

```

        // signal() has no effect during Pickup(), but is important to wake up waiting
        // hungry philosophers during Putdown()
        self[i].signal();
    }
}

init()
{
    // Execution of Pickup(), Putdown() and test() are all mutually exclusive
    //i.e. only one at a time can be executing
    for
        i = 0 to 4

        // Verify that this monitor-based solution is deadlock free and mutually exclusive in that
        //no 2 neighbors can eat simultaneously
        state[i] = thinking;
    }
} // end of monitor

```

We also need to declare **condition self[5];**

- This allows philosopher i to delay herself when she is hungry but is unable to obtain the chopsticks she needs. We are now in a position to describe our solution to the dining-philosophers problem.
- The distribution of the chopsticks is controlled by the monitor Dining Philosophers. Each philosopher, before starting to eat, must invoke the operation pickup().
- This act may result in the suspension of the philosopher process. After the successful completion of the operation, the philosopher may eat.
- Following this, the philosopher invokes the putdown() operation. Thus, philosopher i must invoke the operations pickup() and putdown() in the following sequence:

```

DiningPhilosophers.pickup(i);
...
eat
...
DiningPhilosophers.putdown(i);

```

- It is easy to show that this solution ensures that no two neighbors are eating simultaneously and that no deadlocks will occur. We note, however, that it is possible for a philosopher to starve to death.

Solution of Dining Philosophers Problem

A solution of the Dining Philosophers Problem is to use a semaphore to represent a chopstick. A chopstick can be picked up by executing a wait operation on the semaphore and released by executing a signal semaphore.

The structure of the chopstick is –semaphore chopstick [5];

Initially the elements of the chopstick are initialized to 1 as the chopsticks are on the table and not picked up by a philosopher. The structure of a random philosopher i is given as follows –

```
do {  
    wait( chopstick[i] );  
    wait( chopstick[ (i+1) % 5] );  
    .  
    . EATING THE RICE  
    .  
    signal( chopstick[i] );  
    signal( chopstick[ (i+1) % 5] );  
    .  
    . THINKING  
    .  
} while(1);
```

- In the above structure, first wait operation is performed on chopstick[i] and chopstick[(i+1) % 5]. This means that the philosopher i has picked up the chopsticks on his sides. Then the eating function is performed.
- After that, signal operation is performed on chopstick[i] and chopstick[(i+1) % 5]. This means that the philosopher i has eaten and put down the chopsticks on his sides. Then the philosopher goes back to thinking.

Difficulty with the solution

The above solution makes sure that no two neighboring philosophers can eat at the same time. But this solution can lead to a deadlock. This may happen if all the philosophers pick their left chopstick simultaneously. Then none of them can eat and deadlock occurs.

Some of the ways to avoid deadlock are as follows –

- There should be at most four philosophers on the table.
- An even philosopher should pick the right chopstick and then the left chopstick while an odd philosopher should pick the left chopstick and then the right chopstick.
- A philosopher should only be allowed to pick their chopstick if both are available at the same time.

1.6 Write short notes on Monitors (OR) Explain the structure of monitors. [Nov 2016]

- Monitors are used for process synchronization. With the help of programming languages, we can use a monitor to achieve mutual exclusion among the processes. **Example of monitors: Java Synchronized methods such as Java offers notify() and wait() constructs.**
- In other words, monitors are defined as the construct of programming language, which helps in controlling shared data access.
- The Monitor is a module or package which encapsulates shared data structure, procedures, and the synchronization between the concurrent procedure invocations.

Characteristics of Monitors.

1. Inside the monitors, we can only execute one process at a time.
2. Monitors are the group of procedures, and condition variables that are merged together in a special type of module.
3. If the process is running outside the monitor, then it cannot access the monitor's internal variable. But a process can call the procedures of the monitor.
4. Monitors offer high-level of synchronization
5. Monitors were derived to simplify the complexity of synchronization problems.
6. There is only one process that can be active at a time inside the monitor.

Components of Monitor : There are four main components of the monitor:

1. Initialization
 2. Private data
 3. Monitor procedure
 4. Monitor entry queue
- **Initialization:** – Initialization comprises the code, and when the monitors are created,
 - **Private Data:** – Private data is another component of the monitor. It comprises all the private data, and the private data contains private procedures that can only be used within the monitor. So, outside the monitor, private data is not visible.
 - **Monitor Procedure:** – Monitors Procedures are those procedures that can be called from outside the monitor.
 - **Monitor Entry Queue:** – Monitor entry queue is another essential component of the monitor that includes all the threads, which are called procedures.

Syntax of monitor

```
Monitor Demo // Name of the Monitor  
  
{  
variables;  
condition variables;  
procedure p1 {.....}  
procedure p2 {.....}  
}
```

Condition Variables

There are two types of operations that we can perform on the condition variables of the monitor:

1. Wait
2. Signal

Suppose there are two condition variables condition a, b // Declaring variable

Wait Operation

- **a.wait():** – The process that performs wait operation on the condition variables are suspended and locate the suspended process in a block queue of that condition variable.

Signal Operation

- **a.signal() :** – If a signal operation is performed by the process on the condition variable, then a chance is provided to one of the blocked processes.

Advantages of Monitor

It makes the parallel programming easy, and if monitors are used, then there is less error-prone as compared to the semaphore.

1.7 Demonstrate that monitors and semaphores are equivalent insofar as they can be used to implement the same types of synchronization problems [Apr 2019]

Describe the difference between wait[A], where A is semaphore and B Wait () ,where B is a condition variable in a monitor [Nov 2014]

A semaphore can be implemented using the following monitor code:

```
monitor semaphore {
    int value = 0;
    condition c;

    semaphore_increment() {
        value++;
        c.signal();
    }

    semaphore_decrement() {
        while (value == 0)
            c.wait();
        value--;
    }
}
```

- A monitor could be implemented using a semaphore in the following manner. Each condition variable is represented by a queue of threads waiting for the condition. Each thread has a semaphore associated with its queue entry.
- When a thread performs await operation, it creates a new semaphore (initialized to zero), appends the semaphore to the queue associated with the condition variable, and performs a blocking semaphore decrement operation on the newly created semaphore.
- When a thread performs a signal on a condition variable, the first process in the queue is awakened by performing an increment on the corresponding semaphore.

- Semaphore and Monitor both allow processes to access the shared resources in mutual exclusion. Both are the process synchronization tool. Instead, they are very different from each other.
 - Where Semaphore is an integer variable which can be operated only by wait() and signal() operation apart from the initialization.
 - On the other hand, the Monitor type is an abstract data type whose construct allow one process to get activate at one time.

BASIS FOR COMPARISON	SEMAPHORE	MONITOR
Basic	Semaphores is an integer variable S.	Monitor is an abstract data type.
Action	The value of Semaphore S indicates the number of shared resources availabe in the system	The Monitor type contains shared variables and the set of procedures that operate on the shared variable.
Access	When any process access the shared resources it perform wait() operation on S and when it releases the shared resources it performs signal() operation on S.	When any process wants to access the shared variables in the monitor, it needs to access it through the procedures.
Condition variable	Semaphore does not have condition variables.	Monitor has condition variables.

1.8 Discuss the concept of segmentation with paging in OS [Nov 2017]

Segmented Paging- Segmented paging is a scheme that implements the combination of segmentation and paging to get the best features out of both the techniques. In Segmented Paging, the main memory is divided into variable size segments which are further divided into fixed size pages.

1. Pages are smaller than segments.
2. Each Segment has a page table which means every program has multiple page tables.
3. The logical address is represented as Segment Number (base address), Page number and page offset.

Working-In segmented paging,

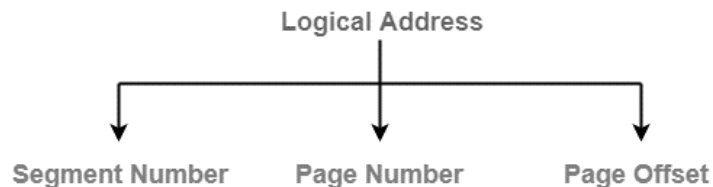
- Process is first divided into segments and then each segment is divided into pages.
- These pages are then stored in the frames of main memory.
- A page table exists for each segment that keeps track of the frames storing the pages of that segment.
- Each page table occupies one frame in the main memory.
- Number of entries in the page table of a segment = Number of pages that segment is divided.
- A segment table exists that keeps track of the frames storing the page tables of segments.
- Number of entries in the segment table of a process = Number of segments that process is divided.
- The base address of the segment table is stored in the segment table base register.

Translating Logical Address into Physical Address-

- CPU always generates a logical address.
- A physical address is needed to access the main memory.

Step-01: CPU generates a logical address consisting of three parts-

1. Segment Number
2. Page Number
3. Page Offset



- Segment Number specifies the specific segment from which CPU wants to read data.
- Page Number specifies the specific page of that segment from which CPU wants to read the data.
- Page Offset specifies the specific word on that page that CPU wants to read.

Step-02:

- For the generated segment number, corresponding entry is located in segment table.
- Segment table provides the frame number of the frame storing the page table of the referred segment.
- The frame containing the page table is located.

Step-03:

- For the generated page number, corresponding entry is located in the page table.
- Page table provides the frame number of the frame storing the required page of the referred segment.
- The frame containing the required page is located.

Step-04:

- The frame number combined with page offset forms the required physical address.
- For the generated page offset, corresponding word is located in the page and read.

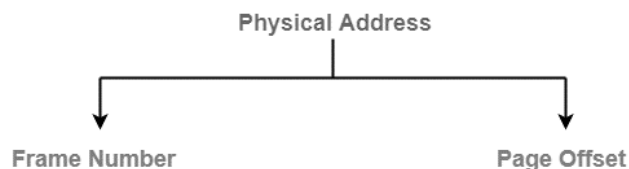
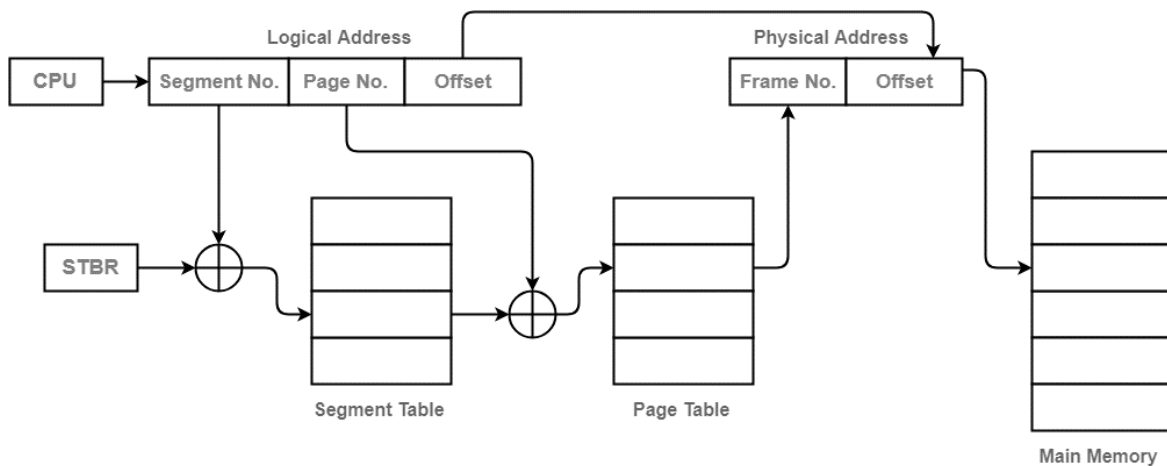


Diagram-The following diagram illustrates the above steps of translating logical address into physical address-



Advantages-

1. It reduces memory usage.
2. Page table size is limited by the segment size.
3. Segment table has only one entry corresponding to one actual segment.
4. External Fragmentation is not there.
5. It simplifies memory allocation.

Disadvantages-

1. Internal Fragmentation will be there.
2. The complexity level will be much higher as compare to paging.
3. Page Tables need to be contiguously stored in the memory.

1.10 Explain Bakery Algorithm OR Multiple process solution (APR'15)

The **Bakery algorithm** is one of the simplest known solutions to the mutual exclusion problem for the general case of N process. Bakery Algorithm is a critical section solution for N processes. The algorithm preserves the first come first serve property.

- Before entering its critical section, the process receives a number. Holder of the smallest number enters the critical section.
- If processes P_i and P_j receive the same number,
 - if $i < j$

P_i is served first;
 - else

P_j is served first.
- The numbering scheme always generates numbers in increasing order of enumeration; i.e., 1, 2, 3, 3, 3, 3, 4, 5, ...

Notation – lexicographical order (ticket #, process id #) – Firstly the ticket number is compared. If same then the process ID is compared next, i.e. -

- $(a, b) < (c, d)$ if $a < c$ or if $a = c$ and $b < d$
- $\max(a[0], \dots, a[n-1])$ is a number, k , such that $k \geq a[i]$ for $i = 0, \dots, n - 1$

Shared data – choosing is an array $[0..n - 1]$ of boolean values; & number is an array $[0..n - 1]$ of integer values. Both are initialized to **False** & **Zero** respectively.

Algorithm Pseudocode –

```
repeat
    choosing[i] := true;
    number[i] := max(number[0], number[1], ..., number[n - 1]) + 1;
    choosing[i] := false;
    for j := 0 to n - 1
        do begin
            while choosing[j] do no-op;
            while number[j] != 0
                and (number[j], j) < (number[i], i) do no-op;
        end;

    critical section
    number[i] := 0;
    remainder section
until false;
```

- Firstly the process sets its “choosing” variable to be TRUE indicating its intent to enter critical section.
- Then it gets assigned the highest ticket number corresponding to other processes.
- Then the “choosing” variable is set to FALSE indicating that it now has a new ticket number.
- The very purpose of the first three lines is that if a process is modifying its TICKET value then at that time some other process should not be allowed to check its old ticket value which is now obsolete.
 - Inside the for loop before checking ticket value we first make sure that all other processes have the “choosing” variable as FALSE.
- After that we proceed to check the ticket values of processes where process with least ticket number/process id gets inside the critical section.
- The exit section just resets the ticket value to zero.

PART [11 Marks]

2.1 What is critical section problem and explain two process solution and multiple process solutions? (APR'15) -

What is critical section problem? How will you find the solution for it? [May 2018]

Refer Questions 1.3, 1.4 and 1.5

2.2 Specify the purpose of Semaphores and its types with an example (APR'16)

[Also Refer Questions 1.7 and 1.5]

What is binary semaphore? How will you implement it?[5] [May 2018]

- **Semaphore** is simply a variable that is non-negative and shared between threads which is used to solve the problem of the critical section in process synchronization.
- A semaphore is a signalling mechanism, and a thread that is waiting on a semaphore can be signalled by another thread. It uses two atomic operations, 1) wait, and 2) signal for the process synchronization.
- The definitions of wait and signal are as follows:

Wait: – In wait operation, the argument ‘S’ value is decrement by 1 if the value of the ‘S’ variable is positive. If the value of the argument variable ‘S’ is zero or negative, no operation is performed.

```
wait (S)
{
while (S<=0);
S--;
}
```

Signal: – In Signal atomic operation, the value of the argument variable ‘S’ is incremented.

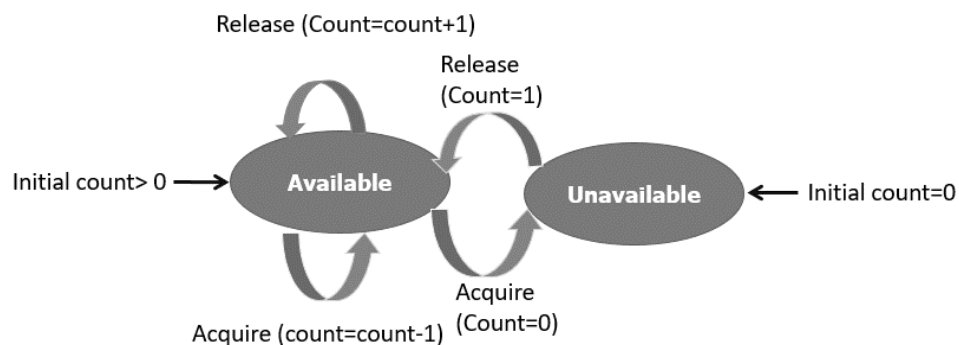
```
Signal (S)
{
S++;
}
```

Characteristics of Semaphore

1. It is a mechanism that can be used to provide synchronization of tasks.
2. It is a low-level synchronization mechanism.
3. Semaphore will always hold a non-negative integer value.
4. Semaphore can be implemented using test operations and interrupts, which should be executed using file descriptors.

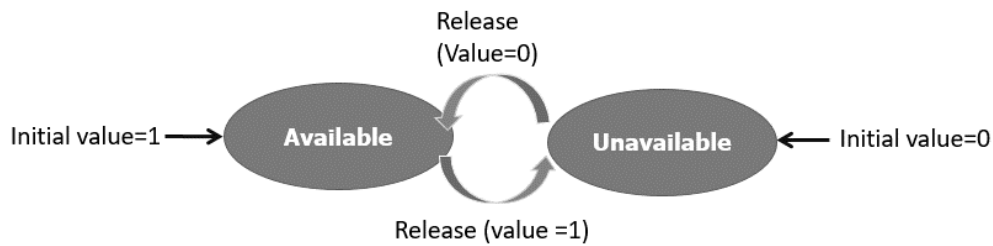
Types of Semaphores

1. **Counting Semaphores:** – Counting Semaphore is defined as a semaphore that contains integer values, and these values have an unrestricted value domain. A counting semaphore is helpful to coordinate the resource access, which includes multiple instances.



If the initial count = 0, the counting semaphore should be created in the unavailable state. However, If the count is > 0, the semaphore is created in the available state, and the number of tokens it has equals to its count.

2. **Binary Semaphores:** – Binary Semaphores are also called Mutex lock. There are two values of binary semaphores, which are 0 and 1. The value of binary semaphore is initialized to 1. We use binary semaphore to remove the problem of the critical section with numerous processes.



In this type of semaphore, the wait operation works only if semaphore = 1, and the signal operation succeeds when semaphore = 0. It is easy to implement than counting semaphores.

Example of Semaphore

Shared var mutex: semaphore = 1;

Process i

begin

.

.

P(mutex);

execute CS;

V(mutex);

.

.

End;

Advantages of Semaphore

1. It allows more than one thread to access the critical section
2. Semaphores are machine-independent.
3. Semaphores are implemented in the machine-independent code of the microkernel.
4. They do not allow multiple processes to enter the critical section.
5. As there is busy waiting in semaphore, there is never a wastage of process time and resources.
6. They are machine-independent, which should be run in the machine-independent code of the microkernel.
7. They allow flexible management of resources.

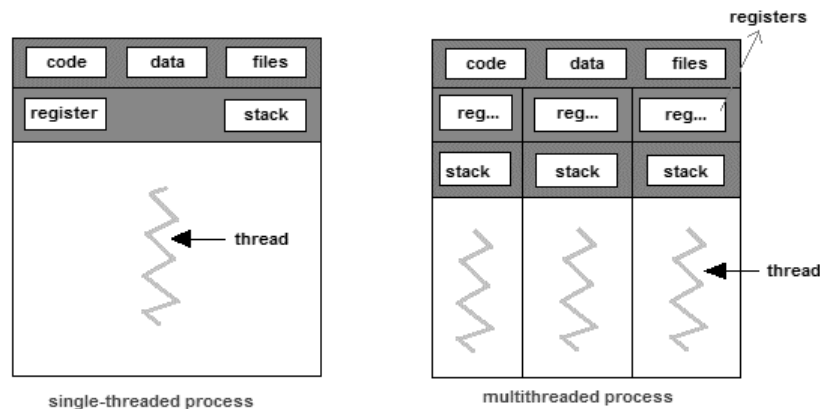
Disadvantages of Semaphore

1. One of the biggest limitations of a semaphore is priority inversion.
2. The operating system has to keep track of all calls to wait and signal semaphore.
3. Their use is never enforced, but it is by convention only.
4. In order to avoid deadlocks in semaphore, the Wait and Signal operations require to be executed in the correct order.
5. Semaphore programming is a complicated, so there are chances of not achieving mutual exclusion.
6. It is also not a practical method for large scale use as their use leads to loss of modularity.
7. Semaphore is more prone to programmer error.
8. It may cause deadlock or violation of mutual exclusion due to programmer error.

2.4 Explain in detail about the threading issues (APR'15)

Thread is an execution unit which consists of its own program counter, a stack, and a set of registers. Threads are also known as Lightweight processes. Threads are popular way to improve application through parallelism. The CPU switches rapidly back and forth among the threads giving illusion that the threads are running in parallel.

As each thread has its own independent resource for process execution, multiple processes can be executed parallelly by increasing number of threads.



Types of Thread

User threads, are above the kernel and without kernel support. These are the threads that application programmers use in their programs.

Kernel threads are supported within the kernel of the OS itself. All modern OSs support kernel level threads, allowing the kernel to perform multiple simultaneous tasks and/or to service multiple kernel system calls simultaneously.

Threading Issues :Following threading issues are:

- The fork() and exec() system call
- Signal handling
- Thread cancelation
- Thread local storage
- Scheduler activation

The fork() and exec() system call

In case if a thread fork is the complete process copied or is the new process single threaded? The answer is here it depends on the system and in case of if new process execs immediately then there is no need to copy all the other thread and if it does not create new process then the whole process should be copied.

Signal Handling

Whenever a multithreaded process receives a signal then to what thread should that signal be conveyed? There are following four main option for signal distribution:

1. Signal deliver to the thread to which the signal applies.
2. Signal deliver to each and every thread in the process.
3. Signal deliver to some of the threads in the process.
4. Assign a particular thread to receive all the signals in a process.

Thread Cancellation

Threads that are no-longer required can be cancelled by another thread in one of two techniques:

1. Asynchronies cancellation
2. Deferred cancellation

Asynchronies Cancellation

It means cancellation of thread immediately. Allocation of resources and inter thread data transfer may be challenging for asynchronies cancellation.

Deferred Cancellation

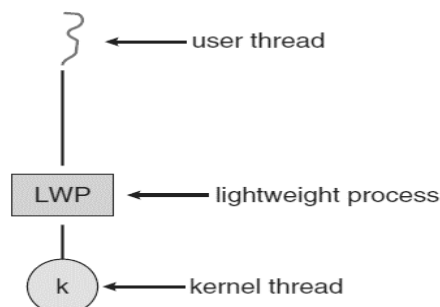
In this method a flag is sets that indicating the thread should cancel itself when it is feasible. It's upon the cancelled thread to check this flag intermittently and exit nicely when it sees the set flag.

Thread Local Storage

The benefit of using threads in the first place is that Most data is shared among the threads but, sometimes threads also need thread explicit data. Major libraries of threads are pThreads, Win32 and java which provide support for thread specific which is called as TLS thread local storage.

Scheduler Activation

Numerous implementation of threads provides a virtual processor as an interface b/w user and kernel thread specifically for two tier model. The virtual processor is called as low weight process (LWP). Kernel thread and LWP has one-to-one correspondence. The available numbers of kernel threads can be changed dynamically. The O.S is used to schedule on to the real system.

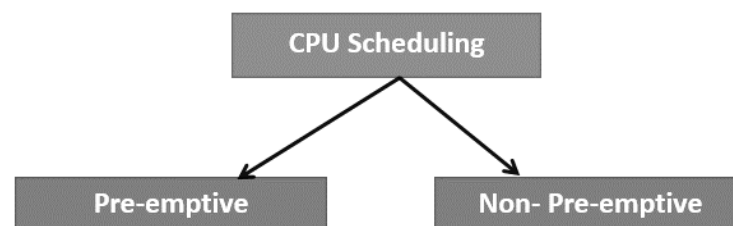
**2.5 Explain the scheduling criteria. With an example explain various CPU Scheduling algorithms [Sep 2020]**

CPU Scheduling is a process of determining which process will own CPU for execution while another process is on hold. The main task of CPU scheduling is to make sure that whenever the CPU remains idle, the OS at least select one of the processes available in the ready queue for execution. The selection process will be carried out by the CPU scheduler. It selects one of the processes in memory that are ready for execution.

CPU Scheduling: Scheduling Criteria : There are many different criterias to check when considering the "best" scheduling algorithm, they are:

- **CPU Utilization**
To make out the best use of CPU and not to waste any CPU cycle, CPU would be working most of the time (Ideally 100% of the time). Considering a real system, CPU usage should range from 40% (lightly loaded) to 90% (heavily loaded.)
- **Throughput**
It is the total number of processes completed per unit time or rather say total amount of work done in a unit of time. This may range from 10/second to 1/hour depending on the specific processes.
- **Turnaround Time**
It is the amount of time taken to execute a particular process, i.e. The interval from time of submission of the process to the time of completion of the process (Wall clock time).
- **Waiting Time**
The sum of the periods spent waiting in the ready queue amount of time a process has been waiting in the ready queue to acquire get control on the CPU.
- **Load Average**
It is the average number of processes residing in the ready queue waiting for their turn to get into the CPU.
- **Response Time**
Amount of time it takes from when a request was submitted until the first response is produced. Remember, it is the time till the first response and not the completion of process execution (final response).
- In general CPU utilization and Throughput are maximized and other factors are reduced for proper optimization.

Types of CPU Scheduling - Here are two kinds of Scheduling methods:



Preemptive Scheduling

- In Preemptive Scheduling, the tasks are mostly assigned with their priorities. Sometimes it is important to run a task with a higher priority before another lower priority task, even if the lower priority task is still running.
- The lower priority task holds for some time and resumes when the higher priority task finishes its execution.

Non-Preemptive Scheduling

- In this type of scheduling method, the CPU has been allocated to a specific process. The process that keeps the CPU busy will release the CPU either by switching context or terminating.

- It is the only method that can be used for various hardware platforms. That's because it doesn't need special hardware (for example, a timer) like preemptive scheduling.

Preemptive or Non-Preemptive Scheduling

To determine if scheduling is preemptive or non-preemptive, consider these four parameters:

1. A process switches from the running to the waiting state.
2. Specific process switches from the running state to the ready state.
3. Specific process switches from the waiting state to the ready state.
4. Process finished its execution and terminated.

Only conditions 1 and 4 apply, the scheduling is called non- preemptive.

All other scheduling are preemptive.

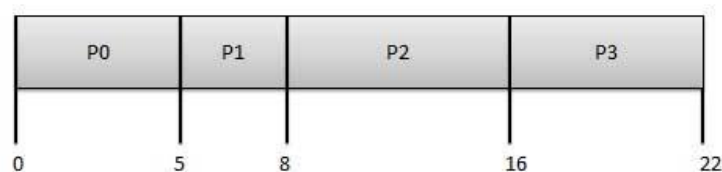
Types of CPU scheduling Algorithm - There are mainly six types of process scheduling algorithms

1. First Come First Serve (FCFS)
2. Shortest-Job-First (SJF) Scheduling
3. Shortest Remaining Time
4. Priority Scheduling
5. Round Robin Scheduling
6. Multilevel Queue Scheduling

First Come First Serve (FCFS)

- Jobs are executed on first come, first serve basis.
- It is a non-preemptive, pre-emptive scheduling algorithm.
- Easy to understand and implement.
- Its implementation is based on FIFO queue.
- Poor in performance as average wait time is high.

Process	Arrival Time	Execute Time	Service Time
P0	0	5	0
P1	1	3	5
P2	2	8	8
P3	3	6	16



Wait time of each process is as follows –

Process	Wait Time : Service Time - Arrival Time
P0	0 - 0 = 0
P1	5 - 1 = 4
P2	8 - 2 = 6
P3	16 - 3 = 13

Average Wait Time: $(0+4+6+13) / 4 = 5.75$

Shortest Job Next (SJN)

- This is also known as **shortest job first**, or SJF
- This is a non-preemptive, pre-emptive scheduling algorithm.
- Best approach to minimize waiting time.
- Easy to implement in Batch systems where required CPU time is known in advance.
- Impossible to implement in interactive systems where required CPU time is not known.
- The processor should know in advance how much time process will take.

Given: Table of processes, and their Arrival time, Execution time

Process	Arrival Time	Execution Time	Service Time
P0	0	5	0
P1	1	3	5
P2	2	8	14
P3	3	6	8

Waiting time of each process is as follows –

Process	Waiting Time
P0	0 - 0 = 0
P1	5 - 1 = 4
P2	14 - 2 = 12
P3	8 - 3 = 5

Average Wait Time: $(0 + 4 + 12 + 5)/4 = 21 / 4 = 5.25$

Priority Based Scheduling

- Priority scheduling is a non-preemptive algorithm and one of the most common scheduling algorithms in batch systems.
- Each process is assigned a priority. Process with highest priority is to be executed first and so on.
- Processes with same priority are executed on first come first served basis.
- Priority can be decided based on memory requirements, time requirements or any other resource requirement.

Given: Table of processes, and their Arrival time, Execution time, and priority. Here we are considering 1 is the lowest priority.

Process	Arrival Time	Execution Time	Priority	Service Time
P0	0	5	1	0
P1	1	3	2	11
P2	2	8	1	14
P3	3	6	3	5

Waiting time of each process is as follows –

Process	Waiting Time
P0	$0 - 0 = 0$
P1	$11 - 1 = 10$
P2	$14 - 2 = 12$
P3	$5 - 3 = 2$

Average Wait Time: $(0 + 10 + 12 + 2)/4 = 24 / 4 = 6$

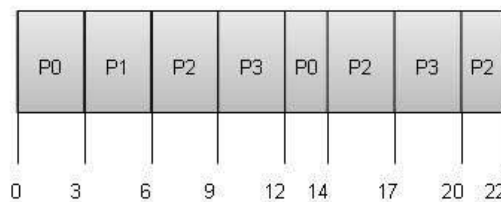
Shortest Remaining Time

- Shortest remaining time (SRT) is the preemptive version of the SJN algorithm.
- The processor is allocated to the job closest to completion but it can be preempted by a newer ready job with shorter time to completion.
- Impossible to implement in interactive systems where required CPU time is not known.
- It is often used in batch environments where short jobs need to give preference.
-

Round Robin Scheduling

- Round Robin is the preemptive process scheduling algorithm.
- Each process is provided a fix time to execute, it is called a **quantum**.
- Once a process is executed for a given time period, it is preempted and other process executes for a given time period.
- Context switching is used to save states of preempted processes.

Quantum = 3



Wait time of each process is as follows –

Process	Wait Time : Service Time - Arrival Time
P0	$(0 - 0) + (12 - 3) = 9$
P1	$(3 - 1) = 2$
P2	$(6 - 2) + (14 - 9) + (20 - 17) = 12$
P3	$(9 - 3) + (17 - 12) = 11$

Average Wait Time: $(9+2+12+11) / 4 = 8.5$

Multiple-Level Queues Scheduling

Multiple-level queues are not an independent scheduling algorithm. They make use of other existing algorithms to group and schedule jobs with common characteristics.

- Multiple queues are maintained for processes with common characteristics.
- Each queue can have its own scheduling algorithms.
- Priorities are assigned to each queue.

For example, CPU-bound jobs can be scheduled in one queue and all I/O-bound jobs in another queue. The Process Scheduler then alternately selects jobs from each queue and assigns them to the CPU based on the algorithm assigned to the queue.